Appendix: Artifact Description/Artifact Evaluation

Artifact Description (AD)

1 Overview of Contributions and Artifacts

1.1 Paper's Main Contributions

C_1 **Low-overhead tracing.** Always-on conventional tracing at ~0.8–2% overhead (vs 18–81% for TAU/dftracer), with Parquet traces 3.6–44× smaller than other formats.

C_2 **Faster post-mortem analysis.** ORCA traces enable post-mortem queries multiple orders of magnitude faster than other analytics-oriented tracers.

C_3 **In-situ analytics.** Flows with operator pushdown to MPI tier execute at sub-2% overhead while reducing data movement by 98.5–99.999%. *Not supported by conventional tracers.*

C_4 **Agentic debugging.** Steerable telemetry lets an LLM agent localize a months-long 4096-rank AMR anomaly in 27 minutes, with 144× volume reduction when reverting to baseline. *Not supported by conventional tracers.*

1.2 Computational Artifacts

A_1 ORCA-Augmented AMR Code
A_2 Post-Mortem Query Scripts
A_3 LLM Prompt for Agentic Workflow

Artifact ID	Contributions Supported	Related Paper Elements
A_1	C_1, C_3, C_4	Figs. 5, 6, 8 (C_1) Fig. 9 and Table 1 (C_3) Fig. 10, live system for A_3 (C_4)
A_2	C_2	Fig. 7
A_3	C_4	Fig. 10

2 Artifact Identification

2.1 Computational Artifact A_1

Relation To Contributions

A_1 is the instrumented BSP workload — Phoebus + Parthenon's Sedov Blast Wave 3D AMR code linked against ORCA — used as the common substrate for three contributions:

- C_1 (Figs. 5, 6, 8): running A_1 under different tracer configurations produces the runtime overhead, trace size, and architectural ablation measurements.
- C_3 (Fig. 9): running A_1 with the in-situ flows from Table 1 produces the pushdown overhead and data-reduction measurements.
- C_4 (Fig. 10): A_1 is the live system that A_3's LLM agent attaches to and steers.

The artifact is provided with two profiles: Mini and Detailed.

- Mini: triggers short runs, sufficient to verify artifact function.
- Detailed: requires 262 nodes on the PDL Wolf cluster and triggers longer experiments, necessary to replicate all results.

Expected Results

- Mini: three timesteps of a simple example program execute, generating Parquet traces via ORCA.
- Detailed: 2000 timesteps of the Blast Wave AMR code, generating Parquet traces via ORCA in two modes—lightweight and detailed. A baseline run is also triggered for overhead comparison.
- Trace sizes: ORCA traces take around 10 B per record, aligning with Fig. 6's measurements.

Expected Reproduction Time (in Minutes)

- Setup: ~30 minutes (building the entire project)
- Execution: ~1 min for Mini, ~2–3 hours for Detailed (Baseline and ORCA only)
- Analysis: <10 minutes

Artifact Setup (incl. Inputs)

Hardware.

- Mini: one multi-core Linux node
- Detailed: 262 nodes on the PDL Wolf cluster

Software.

Required dependencies, must be present in the environment. Versions used by authors are noted in parentheses, other environments may necessitate tweaks to build parameters etc.

- A Linux environment (Ubuntu 22.04)
- An MPI implementation (MVAPICH2 2.3.7)
- C/C++ compiler (GCC 11.4)
- CMake (3.22)
- Rust toolchain (cargo/rustc, stable 1.92)
- TAU (2.34.1)

Dependencies built by orca-umbrella
(https://github.com/pdlfs/orca-umbrella)

ORCA-Augmented AMR Code:

- ORCA — https://github.com/pdlfs/orca
- Phoebus — https://github.com/lanl/phoebus
- Parthenon — https://github.com/parthenon-hpc-lab/parthenon
- amr-tools — https://github.com/pdlfs/amr-tools
- Kokkos 4.0.01 — https://github.com/kokkos/kokkos
- HDF5 1.12.2 — https://github.com/HDFGroup/hdf5

ORCA runtime / data-path:

- Apache Arrow 19.0.0 — https://github.com/apache/arrow
- Snappy 1.2.2 — https://github.com/google/snappy
- Mercury — https://github.com/mercury-hpc/mercury
- libfabric v2.5.1 — https://github.com/ofiwg/libfabric
- DuckDB v1.2.1 — https://github.com/duckdb/duckdb
- yaml-cpp 0.9.0 — https://github.com/jbeder/yaml-cpp
- PSM (forked) — https://github.com/pdlfs/psm

Comparison tracers (for complete results replication):

- DFTracer v2.0.2 (forked) — https://github.com/anku94/dftracer
- Score-P 9.3 (patched, patch automatically applied during build) — https://www.vi-hps.org/projects/score-p/
- Caliper (patched, patch automatically applied during build) — https://github.com/LLNL/Caliper

Datasets / Inputs.

- AMR decks: `blast_wave_3d.<nranks>.pin` (nranks in 512, 1024, 2048, 4096)
- `wfopts.yml`: tuned config for Wolf cluster, only needed for Detailed profile

Installation and Deployment. Preparation commands (indicative, from Ubuntu 22.04, may vary across environments):

```
# Install basic utilities
sudo apt install -y gcc g++ make cmake autoconf \
automake libtool pkg-config git

# Install rust toolchain via rustup (non-interactive, stable)
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs \
| sh -s -- -y --default-toolchain stable
. "$HOME/.cargo/env"
```

Now, orca-umbrella can be built, and will also build the rest of the dependencies in-tree.

```
# Install prefix should be a shared dir on the cluster
OR_PREFIX=/tmp/orca-prefix
OR_BUILD=$OR_PREFIX/build
OR_INSTALL=$OR_PREFIX/install
mkdir -p $OR_PREFIX $OR_BUILD $OR_INSTALL

git clone https://github.com/pdlfs/orca-umbrella.git $OR_PREFIX
cd $OR_BUILD
cmake -DCMAKE_INSTALL_PREFIX=$OR_INSTALL \
        $OR_PREFIX/orca-umbrella
make -j<nproc>
```

Once successfully built, all artifacts will be available in `$OR_INSTALL`. Run the artifact using scripts in `$OR_INSTALL/scripts`.

- Mini: run `run_amr_test.sh`
- Detailed: Generate a valid hostfile as per MPI configuration, with at least 262 nodes, and run:

```
OR_HOSTFILE=/path/to/hostfile \
        $OR_INSTALL/scripts/run_orca_detailed.sh
```

Artifact Execution

- The relevant MPI codes are executed under exhaustive MPI/Kokkos trace collection modes, as per the configured scales and timesteps.
- Detailed: Executes a baseline run and ORCA in lightweight and detailed modes at different scales, and reports overhead.

Artifact Analysis (incl. Outputs)

Each run yields timestep-partitioned Parquet traces in the job directory, organized by schema:

```
parquet/
|-- kokkos_events/
|   `-- ts=<timestep_range>/
|       `-- ranks=<rank_range>.parquet
|-- mpi_collectives/
|   `-- ...
|-- mpi_messages/
|   `-- ...
`-- orca_events/
```

Detailed: Wall-clock times are compared with baseline, and measured runtime overhead of different modes is reported.

2.2 Computational Artifact A_2

Relation To Contributions

A_2 is the post-mortem analytics suite that executes a query suite consisting of three queries, against traces captured by running A_1. The three queries in the suite are: outlier waits, outlier collectives, and timestamp range. It supports C_2 by producing the query-latency comparison in Fig. 7.

Expected Results

The query suite runs against all available traces. Row counts in executed queries are displayed for verification and time taken by queries is logged as a CSV.

Expected Reproduction Time (in Minutes)

- Setup: <5 minutes
- Execution: <1 minute (ORCA only)
- Analysis: <1 minute

Artifact Setup (incl. Inputs)

Hardware. One multi-core Linux node.

Software. ORCA trace queries: Python 3 environment with Polars.

- Python (3.12.11)
- Polars (1.39.3)
- Pandas (2.3.3)

Datasets / Inputs. ORCA traces generated by the AMR codes in A_1.

Installation and Deployment. All Python scripts will be copied to `$OR_INSTALL/scripts/orcaquery` during the orca-umbrella build (see A_1). Polars may be installed by running

`pip install -r requirements.txt`

(or equivalent for the test environment).

Artifact Execution

The script automatically detects all available runs in the suite, and runs each query for each available run.

Artifact Analysis (incl. Outputs)

Matching row counts emitted to stdout for verification of successful query execution, and a CSV + a Pandas dataframe logged to stdout with the query latencies.

2.3 Computational Artifact A_3

Relation To Contributions

A_3 is a prompt that is supplied to the LLM agent (Claude Opus 4.5 at experiment time) to explain how to interact with an active ORCA-augmented BSP code, request telemetry, and schedule in-situ analyses to explain anomalous behavior. It supports C_4 by executing the localization workflow shown in Fig. 10.

Expected Results

(1) (Warmup) LLM is able to interact with the running code, pause, resume, and request and analyze telemetry.

(2) (Exercise) LLM is able to progressively localize the reproduced performance anomaly.
(3) Common case telemetry is two orders of magnitude less than detailed traces.

Caveat: this artifact demonstrates an agentic debugging workflow, not a fixed benchmark. The paper experiment used Claude Opus 4.5, but evaluators may use a comparably capable current coding agent. Exact command trajectories, runtime, and degree of operator intervention may vary, but the intended outcome is the same qualitative result: the agent uses ORCA to localize the anomaly to the same root-cause region. The anomaly itself is specific to our environment.

Expected Reproduction Time (in Minutes)

- Setup: 5 minutes
- Execution: <1 hour
- Analysis: <1 minute

Artifact Setup (incl. Inputs)

Hardware. See A_1.

Software. See A_1, plus:

- Claude Code with a valid Claude subscription or API key — https://www.anthropic.com/claude-code

Datasets / Inputs. A prompt `agentic-workflow.md` describing ORCA and documenting the experimental workflow.

Installation and Deployment. Claude Code installation:

```
curl -fsSL https://claude.ai/install.sh | bash
```

Deployment:

(1) `run_orca_agentic.sh` triggers a 4096-rank AMR run with configuration parameters designed to disable anomaly mitigations and exacerbate occurrences. Simulation starts in paused mode.
(2) Claude Code must be started on another tab, and provided the IP/port of the controller interface of the ORCA controller. It is then asked to initiate the warmup exercise.
(3) After the warmup exercise is complete, the agent should be asked to start the main exercise. Minimal and guided interventions may be necessary in case it makes suboptimal steering choices.

Artifact Execution

- (Warmup) Agent interacts with ORCA and understands the Parquet output structure and analysis workflows.
- (Exercise) Agent uses high-level telemetry to detect the presence of stragglers, and progressively requests additional telemetry by writing custom in-situ flows to identify the root cause (up to `MPI_Wait`, but not beyond).

Artifact Analysis (incl. Outputs)

- (Warmup) Manipulation effects are visible in simulation stdout.
- (Exercise) Agent reports a complete callflow graph of Kokkos regions up to the anomalous `MPI_Wait`.